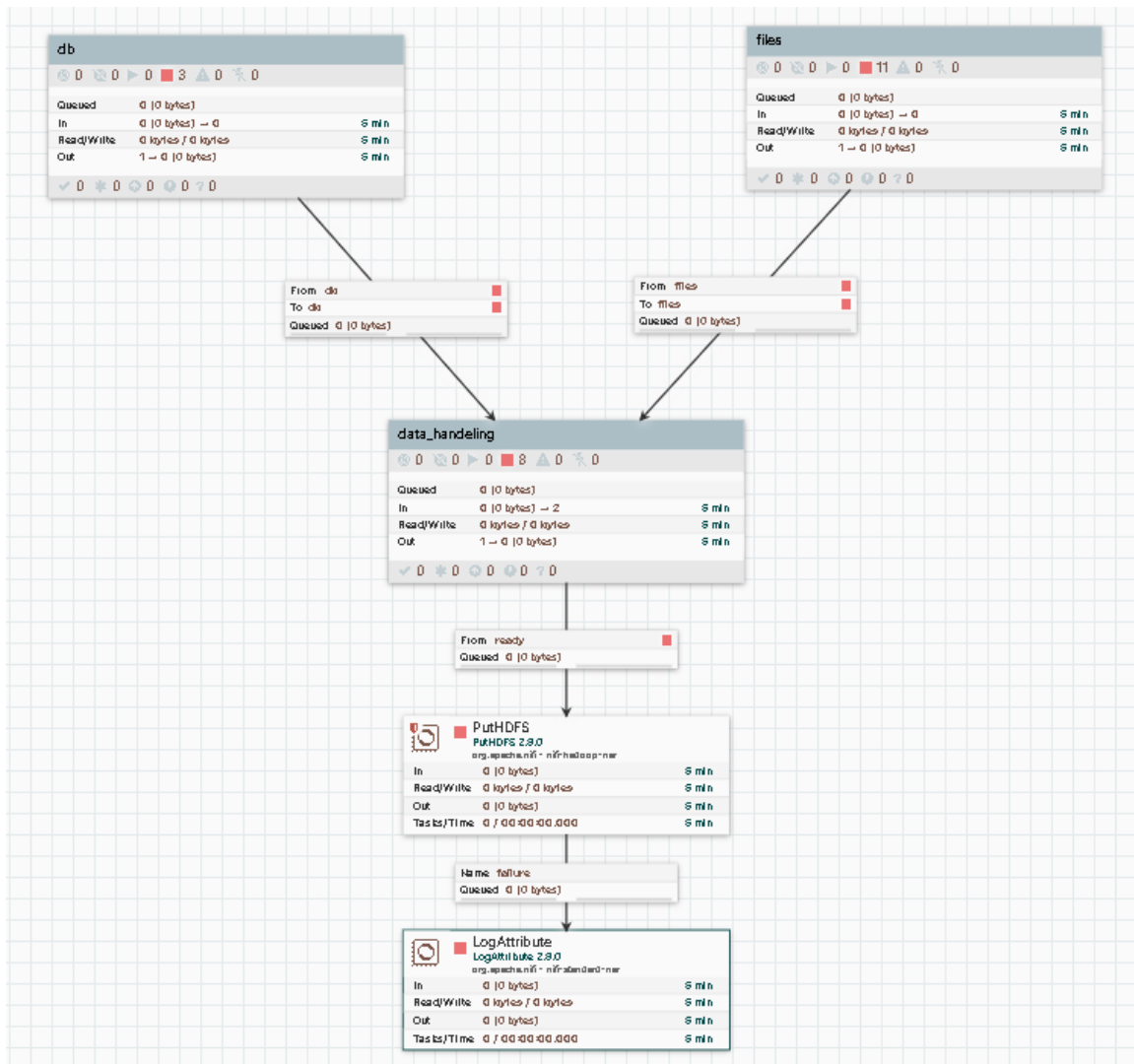


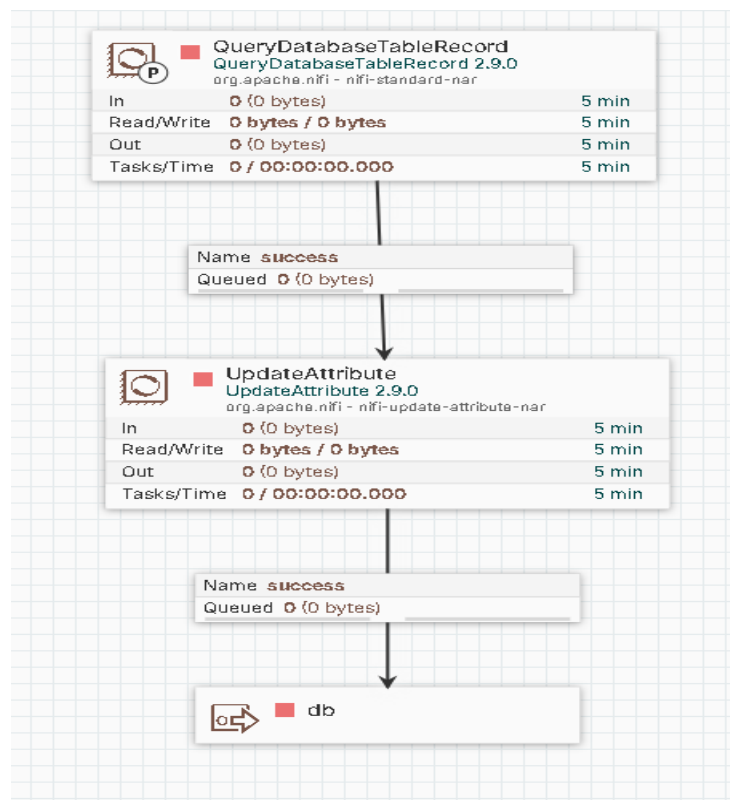
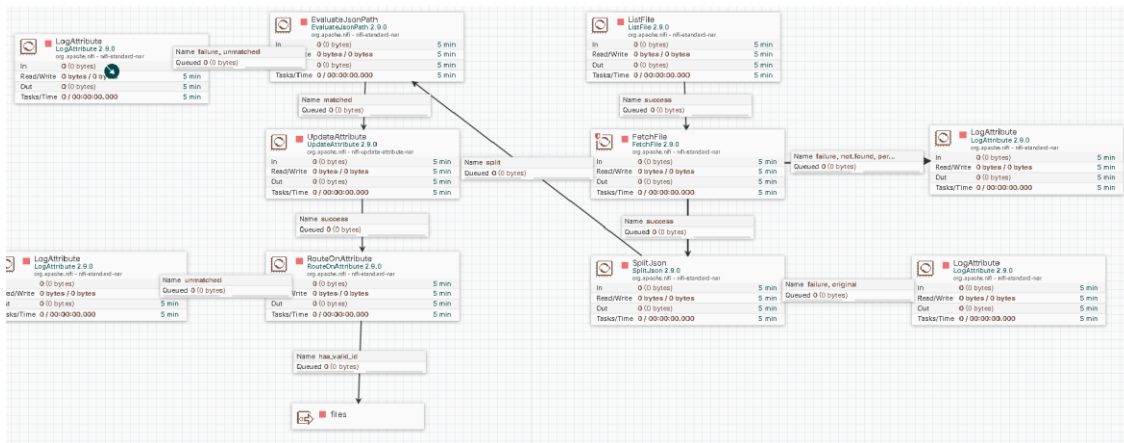
Apache NiFi Assignment: Real-Time Data Ingestion & Processing

Assignment Overview

This project implements a complete data ingestion pipeline using Apache NiFi that simulates real-time streaming data, processes it through multiple transformations, and stores the cleaned results in HDFS (Hadoop Distributed File System). The pipeline handles both file-based and database-based ingestion simultaneously.

Architecture Diagram





Real-Time Data Simulation

Simulation Script (Python)

The simulation script generates continuous JSON data with intentional imperfections to simulate real-world scenarios.

Script Features:

- Generates student records with fields: id, name, email, address, date
- Runs continuously, creating a new JSON file every few seconds
- Introduces data quality issues:
 - Missing IDs (for testing validation)
 - Duplicate records (for deduplication testing)
 - Inconsistent name formats (uppercase, lowercase, mixed case)
 - Missing fields
 - Malformed JSON (occasionally)

Output Location: /opt/nifi/nifi-current/assignment1/nifi_input_data/

python

Simulation script sample structure

while True:

 data = generate_student_records()

 with open(f"students_{timestamp}.json", "w") as f:

 json.dump(data, f)

 time.sleep(interval)

NiFi Pipeline Components

Process Group 1: File Ingestion (files)

Processor	Purpose
ListFile	Lists JSON files from the input directory, tracks processed files to avoid reprocessing
FetchFile	Reads the actual content of each listed file
SplitJson	Splits JSON arrays into individual FlowFiles (one per record)
EvaluateJsonPath	Extracts fields (id, name) as FlowFile attributes for routing decisions
RouteOnAttribute	Routes records based on presence of valid ID (invalid records sent to error log)

Process Group 2: Database Ingestion (db)

Processor	Purpose
QueryDatabaseTableRecord	Extracts data incrementally from PostgreSQL, tracking the maximum ID to only fetch new records

Processor

Purpose

UpdateAttribute

Adds ingestion timestamp to each record

Database Connection Details:

- **Type:** PostgreSQL
- **Table:** simulated_data
- **Incremental Column:** ID (automatically tracked)

Process Group 3: Core Processing (data_handling)

Processor

Purpose

Funnel

Merges data from both sources (files and database) into a single stream

DetectDuplicate

Uses Redis distributed cache to identify duplicate records based on composite key (id + name + email)

UpdateAttribute

Formats data: capitalizes names properly, standardizes date format

MergeContent

Batches records (50 records or 1 minute) into CSV format with header

Final Processing: HDFS Storage

Processor	Purpose
PutHDFS	Writes processed data to HDFS directory
LogAttribute	Captures and logs any write failures

NiFi Best Practices Implemented

1. Back Pressure

FlowFile queues have configured Back Pressure thresholds to prevent overwhelming downstream processors. When a queue reaches the limit, back pressure is applied upstream.

2. Yield Duration

Processors encountering temporary failures yield (pause) for 1 second before retrying, preventing continuous failure loops.

3. Retry & Penalization

FlowFiles that fail processing are penalized and retried (up to 10 retries) with exponential backoff before being routed to failure.

4. Relationship Handling

Each processor properly routes FlowFiles through success/failure relationships, ensuring:

- No data loss (failures are logged but not dropped)
- Proper error tracking
- Clear visibility of processing status

5. Scheduling Strategy

- Database extraction runs every 1 minute on Primary node only
- File listing runs continuously with appropriate run duration settings

6. Execution Node Configuration

- QueryDatabaseTableRecord runs only on Primary node (prevents duplicate database reads)
- Other processors run on All nodes for parallel processing

Data Transformations

Transformation	Processor	Description
Cleaning	RouteOnAttribute	Filters out records missing required fields (ID)
Format Conversion	SplitJson → EvaluateJsonPath	Converts JSON array to individual records with extracted attributes
Name Formatting	UpdateAttribute	Converts names to proper case (e.g., "JOHN" → "John")
Date Standardization	UpdateAttribute	Converts dates to consistent yyyy-MM-dd format
Deduplication	DetectDuplicate	Removes duplicate records using Redis cache
Batching	MergeContent	Batches records into CSV format with header
Enrichment	UpdateAttribute	Adds ingestion timestamp to each record

Storage: HDFS Loading

Target HDFS Path: /user/itversity/clean_optimized_data/

Configuration Requirements:

- Hadoop configuration files (core-site.xml, hdfs-site.xml) must be accessible
- Proper HDFS permissions for the executing user
- Conflict resolution strategy configured (fail on duplicate filenames)

Output Format: CSV with header row

Sample Data Flow

Before Transformation (Raw JSON)

json

```
[
  {"id": 101, "name": "john smith", "email": "john@example.com", "address": "123 Main St", "date": "2024-01-15"},
  {"id": 102, "name": "JANE DOE", "email": "jane@example.com", "address": "456 Oak Ave", "date": "2024-02-20"},
  {"id": null, "name": "test user", "email": "test@example.com", "address": "789 Pine Rd", "date": "2024-03-10"}
]
```

After Transformation (Clean CSV)

csv

id,name,email,address,date

101,John Smith,john@example.com,123 Main St,2024-01-15

102,Jane Doe,jane@example.com,456 Oak Ave,2024-02-20

HadoopOverviewDatanodesDatabase Volume FailuresSnapshotStartup ProgressUtilities

Browse Directory

Asen/ibersalytics/clean_optimized_data

Go

Search:

Show25entries

	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	-rwxr-xr-x	root	root	26.86 KB	May 21 19:16	3	128 MB	file_20260519_221907_2091.json	
☐	-rwxr-xr-x	root	root	18.98 KB	May 21 19:16	3	128 MB	file_20260519_221918_2201.json	
☐	-rwxr-xr-x	root	root	9.56 KB	May 21 19:16	3	128 MB	file_20260519_221938_2391.json	
☐	-rwxr-xr-x	root	root	94.03 KB	May 21 19:16	3	128 MB	file_20260519_221954_2541.json	
☐	-rwxr-xr-x	root	root	13.16 KB	May 21 19:16	3	128 MB	file_20260519_222050_3081.json	
☐	-rwxr-xr-x	root	root	42.14 KB	May 21 19:16	3	128 MB	file_20260519_222100_3181.json	
☐	-rwxr-xr-x	root	root	67.58 KB	May 21 19:16	3	128 MB	file_20260519_222134_3511.json	
☐	-rwxr-xr-x	root	root	91.99 KB	May 21 19:16	3	128 MB	file_20260519_222237_4121.json	
☐	-rwxr-xr-x	root	root	7.54 KB	May 21 19:16	3	128 MB	file_20260519_222326_4591.json	
☐	-rwxr-xr-x	root	root	49.87 KB	May 21 19:57	3	128 MB	file_20260521_195448_21.json	

Showing 1 to 10 of 10 entries

Previous1Next

Hadoop, 2020

 **Real-Time Processing Characteristics**

Feature

Implementation

Continuous Ingestion

ListFile runs continuously, detecting new files immediately

Incremental Database Extraction

QueryDatabaseTableRecord tracks max ID, only fetches new records

Deduplication

Redis cache maintains state across NiFi restarts

Batch Processing

MergeContent batches records for efficient HDFS writes

Error Handling & Data Quality

Scenario	Handling Approach
Missing ID	Routed to LogAttribute, not written to HDFS
Duplicate Record	Detected by Redis cache, discarded (logged)
HDFS Write Failure	Logged, can be routed for retry
Database Connection Failure	Processor yields and retries

Verification Commands

Check HDFS Output

bash

```
hdfs dfs -ls /user/itversity/clean_optimized_data/
```

```
hdfs dfs -cat /user/itversity/clean_optimized_data/*.csv | head -20
```

Check Input Directory

bash

```
ls -la /opt/nifi/nifi-current/assignment1/nifi_input_data/
```

Monitor NiFi Processors

Access NiFi UI: <http://<host>:8080/nifi>

Design Decisions

1. **Two Parallel Ingestion Paths:** File-based and database-based ingestion run independently and merge at the funnel, allowing both real-time files and incremental database changes to be processed together.
2. **Redis for Deduplication:** Chosen over local cache because it maintains state across NiFi restarts and can be shared in a cluster.
3. **SplitJson Before Attribute Extraction:** Splitting early prevents large JSON arrays from monopolizing memory and allows per-record attribute extraction.
4. **MergeContent Batching:** Batching records before HDFS write reduces the number of small files in HDFS, improving storage efficiency.